1. Write a short paragraph to answer these three questions:

- *What are the two major concerns of any software project?*
- *Which of those two do you feel is more important?*
- *Where does the idea of complete functionality fit with these two concerns?*

The 2 biggest concerns of a software project are cost, how much money is required to develop the product, and time, how long it will take to complete. Both of these resources are usually limited, and software developers have to plan within these constraints to create a development schedule that both fits the customer's budget and timeframe for completion. In my opinion, I believe time is more important to a software project as a product can always be scaled down if it's out of budget, but working within the constraints of a deadline is much more restrictive to a development schedule. Complete functionality is ingrained into both of these concerns, as a piece of software may have to compromise on one or both of those parameters in order to achieve its planned functionality; a strict budget or tight schedule may result in reducing the scope of a project or a slower delivery time, or only being able to implement a subset of the planned features.

2. Write a short paragraph to answer these three questions and briefly explain your opinion:

- *In the Agile method for software development, what are the five main phases that occur in each and every iteration?*
- *Do you feel that any of them could be done at the start of the project and not be repeated in every iteration?*
- *Do you feel that would save time overall on the project?*

The Agile method has 5 main phases: planning, design, development, testing, and review. Of those phases, an efficiently planned project could do all of its planning and/or design at the start of the project to minimize time not spent actively developing software later in the life cycle. However, it's also worthwhile to revisit those phases after initial planning/design in case requirements and customer needs change later on in development. While not all phases need to be repeated with the same depth each iteration, they are all equally integral to the development cycle and can't be skipped over to save time overall.

3. Write a short paragraph to answer these four questions and briefly explain your opinion:

- *In the Waterfall method for software development, what are the main phases that occur?*
- *How are they different from the phases in the Agile method?*
- *What other phases are in Waterfall that are left out of Agile?*
- *Do you think these are needed in Waterfall?*
- *Describe a situation using Agile in which one of these extra Waterfall phases might be needed.*

The Waterfall method consists of a requirements, design, implementation, testing, and deployment/maintenance phase taking a project from beginning conceptualization to complete launch. Though the type of tasks covered in both Agile and Waterfall are similar, the ordering of those phases are where the two methods differ; while Agile repeats the cycle of phases for each

iteration of a project, Waterfall places a greater emphasis on completing each phase entirely before moving onto the next for a project. There is a greater emphasis placed on requirements gathering in Waterfall versus Agile, for example, since all expectations and requirements for the final product are meant to be crystallized at this point before moving forward with design and implementation. In order to ensure a project using Waterfall has a clear plan for the rest of development and no confusion/ambiguity on details has the chance to hinder progress later on in implementation, it's important to dedicate extra time to cementing requirements versus an Agile project, which has more flexibility and resistance to unexpected changes later down the line. However, an Agile project can still benefit from an extra phase dedicated to gathering requirements, such as when a development team may need to operate within or work with customers who have strict regulatory policies/requirements.

4. Write one-sentence answers to the following questions:

- *What is a user story?*

A user story is a way of forming a software requirement as a story of how your user is meant to interact with the product you're developing in order to better guide project ideation.

- *What is blueskying?*

Blueskying is the process of brainstorming requirements for your project with the customer, where there are no restrictions on scope and the sky is the limit.

- *What are four things that user stories SHOULD do?*

User stories should 1) describe a single thing the software is meant to do for the customer, 2) using language they understand, 3) be written from the customer's perspective, and 4) be concise.

- *What are three things that user stories SHOULD NOT do?*

User stories shouldn't 1) be overly long or include multiple requirements, 2) use overly technical language, or 3) mention specific technologies.

- *Does the Waterfall method have user stories?*

The Waterfall method doesn't integrate user stories, but they are a part of the Agile method.

5. What is your opinion on the following statements, and why do you feel that way:

- *All assumptions are bad, and no assumption is a good assumption.*
- *A big user story estimate is a bad user story estimate.*

For the purposes of clarifying project details and estimating total project time, it makes sense to interrogate any and all assumptions made by team members to ensure they're either grounded in reality or correctly snuffed out before anything leaves the planning board– even the smallest assumptions can sneak through the assumption-busting phase and make it into the actual project development phase, where they become active risks. As for the statement that a big user story estimate is a bad user story estimate, I don't fully agree that any big user story is useless– it may just need to be refactored into multiple stories that address the same requirements that the initial story mentioned.

6. Fill in the blanks in the statements below, using the following things [you can use each thing for more than one statement]: Blueskying; Role playing; Observation; User story; Estimate; Planning poker.

- *You can dress me up as a use case for a formal occasion:* **user story**
- *The more of me there are, the clearer things become:* **user story**
- *I help you capture EVERYTHING:* **blueskying**
- *I help you get more from the customer:* **role playing**
- *In court, I'd be admissible as firsthand evidence:* **observation**
- *Some people say I'm arrogant, but really I'm just about confidence:* **estimate**
- *Everyone's involved when it comes to me:* ***blueskying***
- *NOTE: when you have finished, check your answers with the result in your text on page 62. Do you agree with the book answers? If you disagree with any of them, justify your preferred answer.*

I agree for the most part with the book's answers, and don't have any statements I feel fit another answer better.

7. Explain what is meant by a better than best-case estimate.

A better than best-case estimate is an unrealistically optimistic estimate of development time, based on ideal conditions and no delay.

8. In your opinion, when would be the best time to tell your customer that you will NOT be able to meet her delivery schedule? Why do you feel that is the best time? Do you think that would be a difficult conversation? If so, how could you make it less difficult?

The best time would be once there's enough reliable evidence showing the deadline is unrealistic. The more delay, the less time for solutions. It's a difficult conversation because customers are usually disappointed or frustrated, so it's best to be transparent, present realistic estimates, and communicate.

9. Write a short paragraph to discuss why you think branching in your software configuration is bad or good, then describe a scenario to support your opinion.

I think branching is good because it allows developers to work on different features without interfering with the main codebase.Keeping branches separate for too long can cause merge conflicts though. For example, one developer can work on a new feature while another fixes separate bugs.

10. Have you used a build tool in your development? If you have, which tool have you used? What are its good points and bad points — in other words, what do you like about it and/or dislike about it?

I've used Vite for web development. I like that it's fast and updates automatically with changes. It was sometimes strange to troubleshoot.